



LeetCode 202: Happy Number

1. Problem Title & Link

Title: LeetCode 202: Happy Number

Link: <https://leetcode.com/problems/happy-number/>

2. Problem Statement (Short Summary)

A **Happy Number** is defined using the following process:

1. Replace the number with the sum of the squares of its digits.
2. Repeat the process.
3. If the number eventually becomes **1**, it is a Happy Number.
4. If the process enters a cycle and never reaches **1**, it is not a Happy Number.

Return:

- true if n is happy
- false otherwise

Example

Input:

n = 19

Process:

$$1^2 + 9^2$$

$$= 1 + 81$$

$$= 82$$

$$8^2 + 2^2$$

$$= 64 + 4$$

$$= 68$$

$$6^2 + 8^2$$

$$= 36 + 64$$

$$= 100$$



$$1^2 + 0^2 + 0^2$$

$$= 1$$

Output:

true

3. Examples (Input → Output)

Example 1

Input:

$$n = 19$$

Output:

true

Example 2

Input:

$$n = 2$$

Output:

false

Explanation:

$$2 \rightarrow 4 \rightarrow 16 \rightarrow 37 \rightarrow 58$$

$$\rightarrow 89 \rightarrow 145 \rightarrow 42$$

$$\rightarrow 20 \rightarrow 4$$

Cycle detected.

Never reaches 1.

Example 3

Input:

$$n = 1$$

Output:

true

Example 4

Input:

$$n = 7$$

Output:

true



4. Constraints

$$1 \leq n \leq 2^{31} - 1$$

Important:

The process may enter a cycle.

Need to detect repeated values.

5. Core Concept (Pattern / Topic)

HashSet / Cycle Detection

Secondary Topics:

- Digit Extraction
- Mathematics
- Floyd Cycle Detection

6. Thought Process (Step-by-Step Explanation)

Understanding the Process

Take:

19

Compute:

$$1^2 + 9^2$$

$$= 82$$

Now repeat:

82

$$8^2 + 2^2$$

$$= 68$$

Continue until:

1

What Problem Can Occur?

Consider:

2

Process:

2

↓

4



↓

16

↓

37

↓

58

↓

89

↓

145

↓

42

↓

20

↓

4

Notice:

4 appears again

From here:

$4 \rightarrow 16 \rightarrow 37 \rightarrow \dots$

repeats forever.

Key Observation

If a number repeats:

Cycle exists

and:

1 will never be reached

Solution Idea

Store every number seen so far.

Before processing:

Have we already seen this number?

If yes:

Cycle found

Return:

false

If number becomes:



1

Return:

true

7. Visual / Intuition Diagram (ASCII Diagram)

Happy Number

19

↓

82

↓

68

↓

100

↓

1

Result:

Happy Number ✓

Non-Happy Number

2

↓

4

↓

16

↓

37

↓



```
58
↓
89
↓
145
↓
42
↓
20
↓
4
Back to:
4
Cycle detected.
Result:
Not Happy ✗
```

8. Pseudocode (Language Independent)

```
create set

while n != 1

    if n exists in set

        return false

    add n to set

    n = sum_of_square_digits(n)

return true
```



Helper Function

```
sum = 0

while n > 0

    digit = n % 10

    sum += digit * digit

    n /= 10

return sum
```

9. Code Implementation

Python

```
class Solution:

    def isHappy(self, n: int) -> bool:

        seen = set()

        while n != 1:

            if n in seen:
                return False

            seen.add(n)

            total = 0

            while n > 0:

                digit = n % 10
                total += digit * digit
                n //= 10

            n = total

        return True
```

Java



```
import java.util.*;

class Solution {

    public boolean isHappy(int n) {

        Set<Integer> seen = new HashSet<>();

        while(n != 1) {

            if(seen.contains(n))
                return false;

            seen.add(n);

            int total = 0;

            while(n > 0) {

                int digit = n % 10;

                total += digit * digit;

                n /= 10;
            }

            n = total;
        }

        return true;
    }
}
```

10. Time & Space Complexity

Time Complexity

$O(\log n)$

Digit extraction requires processing digits.

In practice, the sequence becomes small very quickly.

Space Complexity

$O(\log n)$

For storing previously seen values.



11. Common Mistakes / Edge Cases

Mistake 1

Not detecting cycles.

Example:

$n = 2$

Infinite loop occurs.

Mistake 2

Using:

$\text{digit} * 2$

instead of:

$\text{digit} * \text{digit}$

Need square of digit.

Mistake 3

Forgetting to reset total.

Wrong:

$\text{total} += \dots$

across multiple iterations.

Edge Cases

$n = 1$

Output:

true

Already happy.

$n = 7$

Process:

7

↓

49

↓

97

↓

130

↓



10

↓

1

Happy.

12. Detailed Dry Run

Input:

n = 19

n	Calculation	Result
19	$1^2 + 9^2$	82
82	$8^2 + 2^2$	68
68	$6^2 + 8^2$	100
100	$1^2 + 0^2 + 0^2$	1

Reached:

1

Return:

true

Dry Run for n = 2

n	Result
2	4
4	16
16	37
37	58
58	89
89	145
145	42
42	20
20	4

Already seen:

4

Cycle detected.

Return:

false

13. Common Use Cases (Real-Life / Interview)



Cycle Detection Problems

Happy Number introduces:

Repeated States

Loop Detection

HashSet Usage

Classic example of:

Store visited values

Mathematical Simulation

Repeated transformation until termination.

Interview Purpose

Tests:

HashSet

Digit Extraction

Loop Control

Cycle Detection

14. Common Traps (Important!)

Trap 1

Ignoring repeated values.

Leads to infinite loops.

Trap 2

Not using a HashSet.

Need fast lookup:

$O(1)$

Trap 3

Computing digit sum instead of square digit sum.

Wrong:

$1+9 = 10$

Correct:

$1^2+9^2 = 82$

15. Builds To (Related LeetCode Problems)



LC 202 - Happy Number



LC 141 - Linked List Cycle



LC 142 - Linked List Cycle II



LC 287 - Find Duplicate Number



LC 457 - Circular Array Loop

These teach cycle detection patterns.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
HashSet	$O(\log n)$	$O(\log n)$	Best for Beginners
Floyd Cycle Detection	$O(\log n)$	$O(1)$	Advanced

Approach 1: HashSet (Chosen)

Store every value.

Easy to understand.

Approach 2: Floyd's Cycle Detection

Use:

slow pointer

fast pointer

similar to Linked List Cycle.

No extra space required.

More advanced interview solution.

17. Why This Solution Works (Short Intuition)

The process can only end in two ways:

Reach 1

or

Enter a cycle

By storing previously seen numbers in a HashSet, we can immediately detect cycles and stop the process.

18. Variations / Follow-Up Questions

**Follow-Up 1**

Find how many steps are needed to reach 1.

Example:

19

Steps:

$19 \rightarrow 82 \rightarrow 68 \rightarrow 100 \rightarrow 1$

Answer:

4

Follow-Up 2

Print the entire happy sequence.

Example:

19

Output:

19 82 68 100 1

Follow-Up 3

Use Floyd Cycle Detection instead of HashSet.

Space:

$O(1)$

Follow-Up 4

Check happiness in another number base.

Example:

Base 2

Base 16

Interview Takeaway

Happy Number is one of the best beginner problems for learning:

HashSet

Cycle Detection

Visited-State Tracking

The key realization is:

If a number repeats,

a cycle exists.

And whenever you hear:

Detect repetition



Detect loop

Detect revisited state

you should immediately think:

HashSet / Visited Set